

 RESUMEN COMPARATIVO – CLASES DEL BLOQUE 2

Clase	Dir.	Uso principal
Reader	Lectura	Clase abstracta base para leer caracteres
Writer	Escritura	Clase abstracta base para escribir caracteres
FileReader	Lectura	Leer caracteres de un archivo
FileWriter	Escritura	Escribir caracteres en un archivo
BufferedReader	Lectura	Lectura eficiente por líneas completas
BufferedWriter	Escritura	Escritura eficiente con buffer
File	Gestión	Metadatos, crear, borrar, listar

 MÉTODOS CLAVE DE READER / WRITER

<code>fr.read()</code> lee 1 carácter → int o -1 (fin)	<code>fw.write(str)</code> escribe un String
<code>br.readLine()</code> lee línea completa → String o null	<code>bw.write(str)</code> escribe un String con buffer
<code>bw.newLine()</code> inserta salto de línea del SO	<code>bw.flush()</code> vacía el buffer hacia el archivo
<code>stream.close()</code> cierra (llama a flush automático)	

 MÉTODOS CLAVE DE FILE

<code>exists()</code> ¿existe el archivo/carpetas?	<code>getName()</code> nombre del archivo
<code>getAbsolutePath()</code> ruta completa absoluta	<code>length()</code> tamaño en bytes
<code>createNewFile()</code> crea el archivo (lanza IOException)	<code>delete()</code> borra el archivo o carpeta
<code>mkdir()</code> crea un directorio	<code>mkdirs()</code> crea directorio y padres
<code>List()</code> array de nombres del contenido	<code>listFiles()</code> array de objetos File
<code>isFile()</code> ¿es un archivo?	<code>isDirectory()</code> ¿es una carpeta?

 ERRORES COMUNES

- Usar `FileReader` / `FileWriter` solos para archivos grandes → lento, usar siempre con buffer
- Comparar `readLine()` con `""` en vez de `null` → la línea vacía es `""`, fin de archivo es `null`
- `createNewFile()` sin try/catch → error de compilación (excepción comprobada)
- `File` no lee ni escribe datos — solo gestiona metadatos y rutas. Para leer/escribir usa `FileReader`/`FileWriter`

 EJEMPLO COMPLETO – COPIAR TEXTO LÍNEA A LÍNEA

Patrón más usado en Java real. Combina `BufferedReader` + `BufferedWriter` para copiar eficientemente cualquier archivo de texto.

```
import java.io.*;

public class CopiarTexto {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(
                new FileReader("origen.txt"));
            BufferedWriter bw = new BufferedWriter(
                new FileWriter("destino.txt"));
        } {
            String linea;
            while ((linea = br.readLine()) != null) {
                bw.write(linea);
                bw.newLine();
            }
            System.out.println("Copia realizada");
        } catch (IOException e) {
            System.out.println("Error");
        }
    }
}
```

FLUJO DE DATOS CON BUFFERS



 RUTAS RELATIVAS VS ABSOLUTAS

RELATIVA	ABSOLUTA
<code>File f = new File("archivo.txt");</code> Relativa al directorio de trabajo del proyecto. Más portable.	<code>File f = new File("C:\\Users\\user\\f.txt");</code> Ruta completa. Menos portable entre sistemas operativos.

 PATRONES DE USO FRECUENTES

FILTRAR LÍNEAS QUE CONTIENEN UNA PALABRA

```
String linea;
while ((linea = br.readLine()) != null) {
    if (linea.contains("Java"))
        bw.write(linea); bw.newLine();
}
```

CONVERTIR CONTENIDO A MAYÚSCULAS

```
while ((linea = br.readLine()) != null) {
    bw.write(linea.toUpperCase());
    bw.newLine();
}
```

CONTAR LÍNEAS DE UN ARCHIVO

```
int contador = 0;
while (br.readLine() != null) contador++;
System.out.println("Líneas: " + contador);
```

Texto y Gestión de Archivos

BLOQUE 2 · CHULETARIO · TRÍPTICO A4

BYTES VS CARACTERES

Bloque 1 – Bytes <code>InputStream / OutputStream</code> Datos binarios · 0-255 Archivos de cualquier tipo	Bloque 2 – Caracteres <code>Reader / Writer</code> Texto Unicode Archivos .txt, .csv...
--	---

Jerarquía de Clases

```
java.io
├ Reader                + abstracta
│   ├── FileReader      + leer char de archivo
│   │   └─ BufferedReader + leer líneas (envuelve Reader)
│   └─ Writer          + abstracta
│       ├── FileWriter  + escribir char en archivo
│       │   └─ BufferedWriter + escribir eficiente
│       └─ File         + gestión (no lee/escribe datos)
```

CONTENIDOS DE ESTE TRÍPTICO

- Interior **izq.** — `FileReader`, `FileWriter`, desglose línea a línea
- Interior **cent.** — `BufferedReader`, `BufferedWriter`, clase `File`
- Interior **der.** — Directorios, rutas, tabla resumen completa
- Solapa — Ejemplo de copia completa, rutas, patrones frecuentes
- Contraportada — Tabla de clases, métodos clave, errores comunes

REGLA DE ORO DEL BLOQUE 2

Usa siempre `BufferedReader` / `BufferedWriter` envolviendo a `FileReader` / `FileWriter`. La lectura por líneas con `readLine()` es la forma más habitual en Java real.

CARA A → Exterior · CARA B → Interior (teoría y referencia)
Imprimir: A4 horizontal · doble cara · voltear por lado largo

🔥 FLUJOS DE CARACTERES – READER Y WRITER

- Reader / Writer** trabajan con caracteres Unicode. Ideales para archivos de texto (.txt, .csv). A diferencia de bytes, gestionan correctamente codificación de caracteres.
- **Reader** → leer caracteres · **Writer** → escribir caracteres
 - Son **clases abstractas** → se usan sus subclases
 - **read()** devuelve el carácter como **int** o **-1** al final

📄 FILEREADER – LEER TEXTO DE UN ARCHIVO

```
import java.io.FileReader;
import java.io.IOException;

public class LeerTexto {
    public static void main(String[] args) {
        try (FileReader fr =
            new FileReader("archivo.txt")) {
            int character;
            while ((character = fr.read()) != -1) {
                System.out.print((char) character);
            }
        } catch (IOException e) {
            System.out.println("Error al leer");
        }
    }
}
```

Código	Qué hace
<code>new FileReader("f")</code>	Abre archivo para leer caracteres
<code>fr.read()</code>	Lee 1 carácter → int (valor Unicode) o -1
<code>≠ -1</code>	Condición de fin de archivo
<code>(char) caracter</code>	Convierte el int al carácter real

🔥 FILEWRITER – ESCRIBIR TEXTO EN UN ARCHIVO

```
import java.io.FileWriter;
import java.io.IOException;

public class EscribirTexto {
    public static void main(String[] args) {
        try (FileWriter fw =
            new FileWriter("archivo.txt")) {
            fw.write("Hola alumnos\n");
            fw.write("Segunda línea");
        } catch (IOException e) {
            System.out.println("Error al escribir");
        }
    }
}
```

MODO APPEND – AÑADIR SIN BORRAR

```
// true = añadir al final, no sobrescribir
FileWriter fw = new FileWriter("archivo.txt", true);
```

FileReader / FileWriter solos son ineficientes: leen/escriben carácter a carácter. Envuélvelos siempre con **Buffered***.

⚡ BUFFEREDREADER – LECTURA EFICIENTE POR LÍNEAS

Envuelve a **FileReader** y añade un buffer interno. Permite leer líneas completas con **readLine()**. Es la forma **más usada en Java real**.

```
import java.io.*;

public class LeerLineas {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
            new FileReader("archivo.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error al leer");
        }
    }
}
```

Código	Qué hace
<code>new BufferedReader(new FileReader("f"))</code>	Encapsula el FileReader con buffer
<code>br.readLine()</code>	Lee una línea completa como String
<code>≠ null</code>	Fin de archivo cuando devuelve null (no -1)

⚡ BUFFEREDWRITER – ESCRITURA EFICIENTE

```
try (BufferedWriter bw = new BufferedWriter(
    new FileWriter("archivo.txt"))) {
    bw.write("Primera línea");
    bw.newLine(); // salto de línea del SO
    bw.write("Segunda línea");
} catch (IOException e) { ... }
```

newLine() inserta el salto de línea correcto del sistema operativo (**\r\n** en Windows, **\n** en Linux/Mac). Más seguro que escribir **"\n"** manualmente.

📁 LA CLASE FILE – GESTIÓN DE ARCHIVOS Y CARPETAS

File **no** lee ni escribe datos. Solo gestiona metadatos, rutas, y operaciones del sistema de archivos.

```
import java.io.File;
File archivo = new File("archivo.txt");
```

CONSULTAR INFORMACIÓN

```
System.out.println(archivo.exists());           // true/false
System.out.println(archivo.getName());           // "archivo.txt"
System.out.println(archivo.getAbsolutePath());   // ruta completa
System.out.println(archivo.length());            // tamaño bytes
```

CREAR Y BORRAR ARCHIVOS

```
File nuevo = new File("nuevo.txt");
try {
    nuevo.createNewFile(); // ⚠ lanza IOException
} catch (IOException e) { ... }
nuevo.delete();           // borra sin excepción
```

📁 TRABAJAR CON DIRECTORIOS

```
File carpeta = new File("miCarpeta");

// Crear directorio simple
carpeta.mkdir();

// Crear directorio y todos sus padres
new File("a/b/c").mkdirs();

// Listar nombres del contenido
String[] nombres = carpeta.list();
for (String n : nombres)
    System.out.println(n);

// Listar como objetos File
File[] archivos = carpeta.listFiles();
for (File f : archivos) {
    System.out.println(f.getName()
        + (f.isDirectory() ? " [DIR]" : ""));
}
```

CREAR CARPETA Y ARCHIVO DENTRO

```
File dir = new File("datos");
dir.mkdir();
File f = new File(dir, "info.txt"); // datos/info.txt
try { f.createNewFile(); } catch (IOException e) {}
```

⚠ FILEREADER VS FILEINPUTSTREAM – ¿CUÁNDO USAR CADA UNO?

	FileReader	FileInputStream
Tipo de dato	Caracteres (Unicode)	Bytes binarios
Ideal para	.txt, .csv, .html	Imágenes, audio, cualquier archivo
Unidad de lectura	char (2 bytes)	byte (1 byte)
Fin de archivo	<code>-1 / null</code>	<code>-1</code>
Versión con buffer	<code>BufferedReader</code>	<code>BufferedInputStream</code>

📄 TABLA RESUMEN DEL BLOQUE

Operación	Clase	Método clave
Leer carácter	<code>FileReader</code>	<code>read()</code> → int
Escribir texto	<code>FileWriter</code>	<code>write(String)</code>
Leer línea	<code>BufferedReader</code>	<code>readLine()</code> → String
Escribir línea	<code>BufferedWriter</code>	<code>write()</code> + <code>newLine()</code>
Fin de archivo (char)	—	<code>read() = -1</code>
Fin de archivo (línea)	—	<code>readLine() = null</code>
Modo append	<code>FileWriter</code>	constructor(true)
Crear archivo	<code>File</code>	<code>createNewFile()</code>
Borrar archivo	<code>File</code>	<code>delete()</code>
Crear carpeta	<code>File</code>	<code>mkdir()</code> / <code>mkdirs()</code>
Listar carpeta	<code>File</code>	<code>list()</code> / <code>listFiles()</code>

Patrón correcto: `BufferedReader(new FileReader("f")) + while(readLine() != null)`. Nunca uses **FileReader** solo para leer texto real.